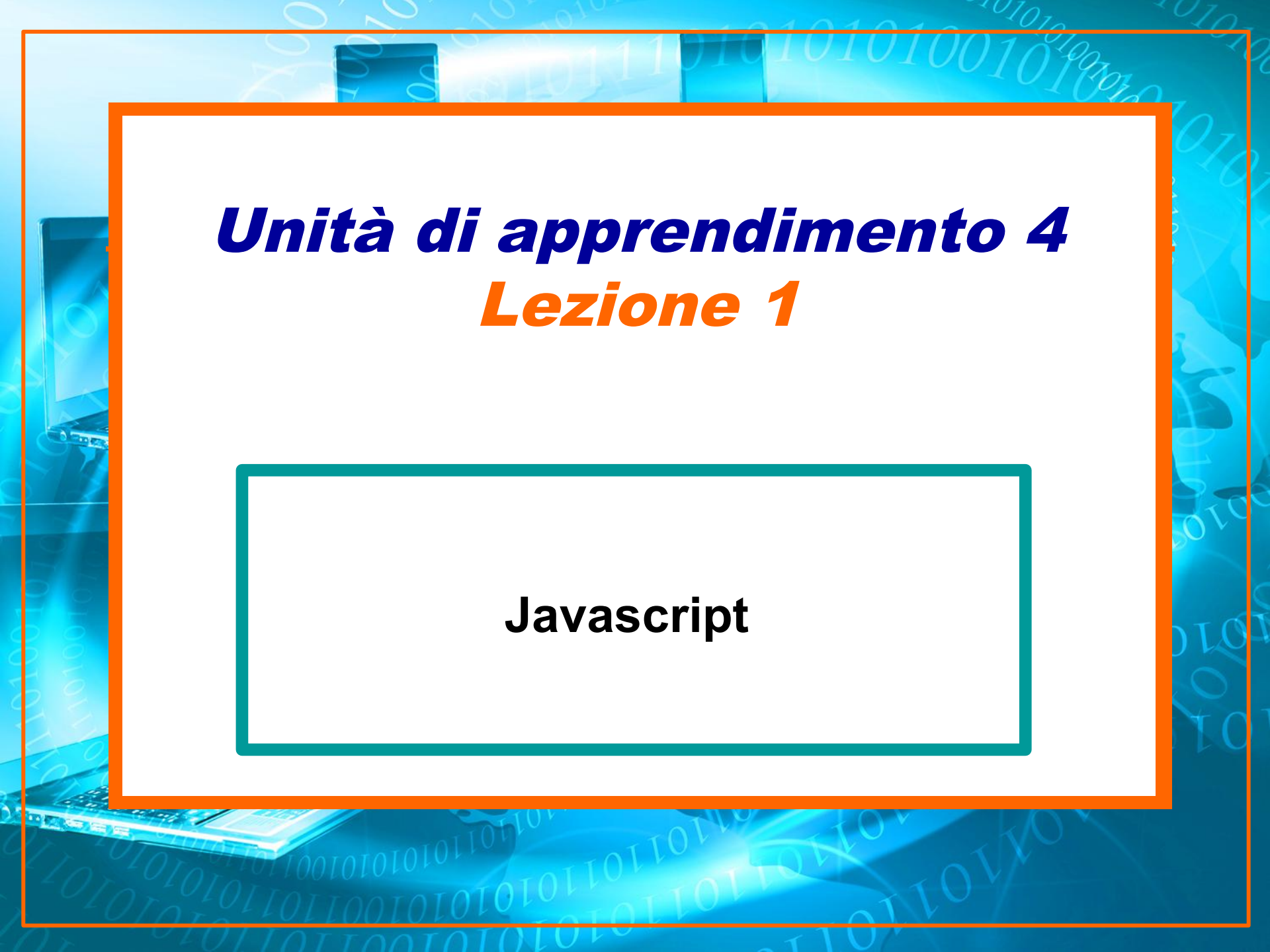


Unità di apprendimento 4

**Applicazioni WEB
JavaScript e Ajax**



The background of the slide is a blue gradient with a pattern of binary code (0s and 1s) in a lighter blue color. On the left side, there is a partial view of a laptop screen and keyboard.

Unità di apprendimento 4

Lezione 1

Javascript

Generalità

- **JavaScript** è uno dei linguaggi di programmazione più usati al mondo
- È stato ideato nel 1995 da Netscape e rilasciato con la versione 2.0 del suo browser, Netscape Navigator, dapprima con il nome LiveScript e subito dopo con l'attuale nome
- L'enorme diffusione di JavaScript è dovuta principalmente al fiorire di numerose librerie nate allo scopo di semplificare la programmazione sul browser, ma anche alla nascita di framework lato server e nel mondo mobile che lo supportano come linguaggio principale

Generalità

- JavaScript aggiunte alle pagine HTML la possibilità di essere modificate in modo dinamico, in base all'interazione dell'utente con il browser (lato client)
- Come per qualsiasi linguaggio di programmazione, per iniziare a lavorare con JavaScript c'è bisogno di almeno tre strumenti:
 - un editor
 - un interprete
 - un debugger
- Come editor è naturalmente possibile utilizzare un comune editor di testo

ECMAScript

- **ECMAScript** è una specifica che definisce le **regole** e le **caratteristiche** di un linguaggio di scripting
- Definisce la sintassi, i comportamenti di base, gli oggetti e le funzionalità che un linguaggio di scripting dovrebbe avere, ma non è un linguaggio vero e proprio
- Ogni nuova versione di ECMAScript aggiunge nuove caratteristiche e funzionalità che poi vengono adottate da linguaggi come JavaScript
- JavaScript può includere anche funzionalità aggiuntive o specifiche che non sono parte dello standard ECMAScript (manipolazione del DOM, API, gestione di eventi, Garbage collection, ...)

ECMAScript

- ECMAScript 1 (1997), stabilisce le basi per la sintassi e il comportamento del linguaggio, definizione di variabili (var), tipi di dati, operatori e strutture di controllo
- ECMAScript 2 (1998), aggiornamenti minori
- ECMAScript 3 (1999), metodi degli oggetti, espressioni regolari, try/catch, gestione delle stringhe
- ECMAScript 4, mai completato
- ECMAScript 5 (2009), strict mode, JSON, altri metodi degli oggetti, getter/setter

ECMAScript

- ECMAScript 6 (2015), aggiornamento radicale:
 - nuova definizione di variabili (let con scope di blocco e const)
 - arrow functions (funzioni anonime per una sintassi più concisa)
 - template literals (stringhe con interpolazione di variabili)
 - sintassi basata su classi
 - codice modulare (import, export)
 - promise (gestione asincrona del codice)
 - rest e spread operators (per raccogliere più valori in un'unica variabile o espandere i valori di più variabili in un'unica variabile)
 - funzione generatore (function* e yield, funzione che può essere sospesa restituendo il controllo al chiamante, utile quando si vogliono restituire più valori nel tempo, ma utilizzandone uno alla volta e quando se ne ha bisogno)

ECMAScript

- ECMAScript 7 (2016), operatore esponente (**)
- ECMAScript 8 (2017), async/await (codice asincrono più leggibile), Object.entries() e Object.values() (per ottenere le coppie chiave/valore o solo i valori di un oggetto)
- ECMAScript 9 (2018), Rest/Spread per oggetti
- ECMAScript 10 (2019), novità minori
- ECMAScript 11 (2020), operatore di coalescenza null (operatore ?? che restituisce il primo valore non null/undefined tra due operandi)
- ECMAScript 12 (2021), novità minori

ECMAScript

- ECMAScript 13 (2022), attributi privati per le classi (`#campoPrivato`), uso di `await` fuori da una funzione asincrona
- ECMAScript 14 (2023), novità minori
- ECMAScript 15 (2024), novità minori
- ECMAScript 16 (2025), novità minori

Inserire codice JavaScript

- Esistono essenzialmente **tre modi** per inserire codice JavaScript in una pagina HTML:
 - **inserire codice inline, associato a un evento**: consiste nell'inserire direttamente le istruzioni JavaScript nel codice di un elemento HTML, assegnandolo ad un attributo che rappresenta un evento. Chiariamo il concetto con un esempio:

```
<button type="button"
onclick="alert('Ciao!')">Cliccami</button>
```
 - Abbiamo assegnato all'attributo **onclick** dell'elemento button la stringa alert('Ciao!').
 - L'attributo onclick rappresenta l'evento del clic sul pulsante del mouse, quindi in corrispondenza di questo evento verrà analizzato ed eseguito il codice JavaScript assegnato. Nel caso specifico verrà visualizzato un box con la scritta Ciao!

Inserire codice JavaScript

- **inserire codice inline, associato a un link:** consiste nell'inserire direttamente le istruzioni JavaScript nel codice di un elemento HTML di tipo ancora, assegnandolo all'attributo che rappresenta il collegamento a una pagina di destinazione:
`<a href="javascript:alert('Ciao!')">Cliccami</a>`
- indichiamo al browser di interpretare il link come richiesta di esecuzione del codice JavaScript specificato invece che come un collegamento ad un'altra pagina

Inserire codice JavaScript

- **scrivere blocchi di codice nella pagina:** quando il codice da eseguire è più complesso o abbiamo necessità di definire variabili e funzioni o semplicemente troppo lungo e quindi poco gestibile e leggibile. In questi casi possiamo ricorrere al tag **<script>** per inserire blocchi di codice JavaScript in una pagina HTML, come nel seguente esempio:

```
<script>alert('Ciao!')</script>
```

- Quando il parser HTML del browser esamina la pagina, riconosce il tag `<script>` come blocco di codice e ne passa il contenuto all'engine JavaScript, che lo esegue. Possiamo inserire blocchi di codice (e i relativi tag `<script>`) nella sezione `<head>` o nella sezione `<body>` della pagina HTML

Inserire codice JavaScript

- **importare file con codice JavaScript esterno:** consiste nel collegare alla pagina HTML, codice JavaScript presente in un file esterno. Questa tecnica permette di agganciare script e librerie in modo detto non intrusivo, con il vantaggio di una separazione netta tra la struttura del documento e il codice. Per inserire un file JavaScript esterno ci serviamo sempre del tag `<script>` in cui specificando l'attributo **src**:

```
<script src="codice.js"></script>
```

- **Attenzione!** Se il codice JavaScript interagisce con un elemento HTML, occorre assicurarsi che tale elemento sia già stato **analizzato** dal parser HTML: così il corrispondente oggetto sarà già disponibile in memoria. Ecco perchè quasi sempre è meglio inserire codice JavaScript **in fondo** alla pagina prima della chiusura del tag `</body>`

Sintassi

- Considereremo JavaScript basato su ECMAScript 5 e precedenti
- Il codice JavaScript è composto da una sequenza di istruzioni che viene interpretata ed eseguita dall'engine
- In questa sequenza, ciascuna istruzione (o blocco di istruzioni) è delimitata da un **punto e virgola (;)**

```
var x = 5;
```

```
x = x + 1;
```

- il punto e virgola **non è obbligatorio**, tuttavia è buona norma inserire sempre la punteggiatura di terminazione per **evitare ambiguità e risultati inattesi**
- mettere il punto e virgola in fondo ad ogni istruzione ci consente di scrivere **più istruzioni sulla stessa riga**

Sintassi

- Javascript è **case sensitive**
- JavaScript prevede delle sequenze di caratteri per inserire commenti nel codice
 - tutto ciò che viene marcato come commento non viene preso in considerazione dall'interprete JavaScript
 - il commento a singola riga inizia con i caratteri //
 - il commento multiriga (multiline) prevede la sequenza iniziale /* (slash e asterisco) e si conclude con */ (asterisco e slash): tutto ciò che viene inserito all'interno di questa coppia di terminazioni sarà considerato un commento, indipendentemente dal numero di righe utilizzate

I tipi di dati

- JavaScript prevede sei tipi di dato primitivi:
 - numeri
 - stringhe
 - booleani
 - null
 - undefined
 - oggetti, un tipo di dato complesso
- Tutti gli altri elementi previsti dal linguaggio, come ad esempio gli array, le espressioni regolari, le funzioni, sono in realtà oggetti

Stringhe

- In JavaScript una **stringa** è una sequenza di caratteri delimitata da doppi o singoli apici
`"stringa numero uno"`
`'stringa numero due'`
 - per inserire caratteri speciali all'interno di una stringa si fa ricorso al carattere di escaping \

Numeri

- JavaScript ha un **unico** tipo di dato **numerico**, cioè non c'è distinzione formale, ad esempio, tra intero e decimale
- Internamente tutti i valori numerici sono rappresentati come numeri in virgola mobile, ma se non è specificata la parte decimale il numero viene trattato come intero

```
var interoNegativo = -10;  
var zero = 0;  
var interoPositivo = 123;
```

- Per rappresentare un numero non intero facciamo ricorso al punto come separatore della parte decimale:

```
var numeroDecimale = 0.52;  
var decimaleNegativo = -1.2;  
var decimaleZero = 1.0;
```

Numeri

- Oltre alla classica notazione in base dieci, possiamo rappresentare i numeri in notazione **ottale** ed **esadecimale**:

```
var numeroOttale = 0123; //equivalente a 83
```

```
var numeroEsadecimale = 0x123; //equivalente a 291
```

- Un numero ottale inizia con lo zero mentre un numero esadecimale inizia con zero e x (0x)
- L'insieme dei numeri rappresentabili in JavaScript cade nell'intervallo compreso tra
$$-1.79769 \cdot 10^{308} \text{ e } 1.79769 \cdot 10^{308}$$
- in quanto i numeri in Javascript sono a 64bit

Numeri

- Ogni valore che va al di fuori dell'intervallo rappresentabile non genera un'eccezione ma viene rappresentato da due valori speciali:
 - **Infinity**, detto anche infinito positivo
 - **-Infinity**, detto infinito negativo
- Un altro valore numerico speciale è **NaN**, acronimo di Not a Number, che indica un valore numerico non definito
 - nel seguente esempio il valore della variabile x dopo l'assegnamento è NaN:

```
var x = x + 1;
```

Null e Undefined

- Il tipo di dato null prevede il solo valore **null** e non rientra tra nessuno degli altri tipi di dato del linguaggio

```
var x = null;
```

- Attenzione! Il valore null è diverso dal valore numerico 0 o dalla stringa vuota!

- Il tipo di dato **undefined** prevede il solo valore undefined e rappresenta un valore che non esiste

```
var x;
```

- è il valore di una variabile non inizializzata, ovvero a cui non è stato assegnato nessun valore, nemmeno null!

Tipizzazione debole

- JavaScript è un linguaggio a **tipizzazione debole** o **dinamica**, quando dichiariamo una variabile non ne specifichiamo il tipo di dato
- Dunque il tipo di dato che può contenere una variabile **può cambiare** durante l'esecuzione dello script
- Quando dichiariamo una variabile senza specificare un valore, a questa viene assegnato il valore undefined

```
var miaVariabile; // undefined  
miaVariabile = 1;  
miaVariabile = null;  
miaVariabile = "uno";  
miaVariabile = true;
```

Gli operatori

- Gli operatori sono simboli o parole chiave che permettono di eseguire operazioni su valori o variabili
- Sono gli strumenti che ci permettono di fare calcoli, confronti o manipolare dati
- Possiamo dividere gli operatori in tre categorie principali:
 - unari, lavorano su un solo valore
 - binari, richiedono due valori e sono i più comuni
 - ternari, più rari e operano su tre valori

Gli operatori

- Gli operatori **unari** sono utili quando vogliamo modificare, invertire o valutare un singolo dato senza confrontarlo con un altro

```
var x = 5;  
+x; // forza un valore a numero  
-x; // inverte il segno di un numero  
x++; // incrementa di 1  
x--; // decrementa di 1  
!x; // negazione logica
```

Gli operatori

- Gli operatori binari sono i più usati nella programmazione perché ci permettono di fare calcoli, confronti o combinazioni logiche tra due dati

- aritmetici, per fare calcoli matematici

```
var a = 5, b = 2;  
a + b; // somma  
a - b; // sottrazione  
a * b; // moltiplicazione  
a / b; // divisione  
a % b; // resto
```

- in caso di stringhe il + effettua una concatenazione

```
var a = "Mario", b = "Rossi";  
var c = a + ' ' + b; // "Mario Rossi"
```

Gli operatori

- di confronto, per confrontare valori e restituire true o false

```
a > b;    // a maggiore di b
a < b;    // a minore di b
a >= b;   // a maggiore o uguale a b
a <= b;   // a minore o uguale a b
a == b;   // a uguale a b
a != b;   // a diverso da b
```

- logici, per concatenare condizioni

```
x && b;   // x and b
x || b;   // x or b
```

- di assegnamento, per assegnare o aggiornare valori

```
c = 10;
c += 5;    // equivalente a c = c + 5
c -= 3;    // equivalente a c = c - 3
...
```

Gli operatori

- L'operatore ternario è un operatore speciale che opera su tre valori ed è una scorciatoia per scrivere delle brevi istruzioni dipendenti da delle condizioni in una sola riga
- Si utilizza:
 - quando serve una scelta semplice tra due valori
 - quando si vuole scrivere codice soggetto a condizioni in maniera più compatta
- Per condizioni complesse o più ramificazioni è meglio usare il costrutto tradizionale (che vedremo più avanti insieme a una applicazione dell'operatore ternario) perché in quei casi risulta più leggibile

Uguaglianza e disuguaglianza

- Gli operatori di uguaglianza e disuguaglianza **debole** (senza coercizione di tipo) effettuano automaticamente una conversione prima di confrontare gli operandi, ossia provano a “rendere compatibili” i valori
- Sono rappresentati con **==** e **!=**

```
" " == 0           // true
```

```
"0" == false      // true
```

```
[] == false       // true
```

Uguaglianza e disuguaglianza

- Gli operatori di uguaglianza e disuguaglianza **stretta** (con coercizione di tipo) confrontano gli operandi senza effettuare alcuna conversione
- Sono rappresentati con **===** e **!==**

```
var c = 0;  
var b = "0";  
c == b    // true  
c === b   // false
```

- quindi due espressioni vengono considerate uguali soltanto se sono dello stesso tipo e rappresentano effettivamente lo stesso valore

Conversione di stringhe

- Quando non si può evitare il confronto tra tipi diversi è opportuno convertire esplicitamente un valore di un tipo in un altro tipo, ricorrendo ad alcune funzioni predefinite
- La funzione **parseInt()** converte una stringa numerica in un valore intero e prevede due parametri:
 - il primo è la stringa da convertire
 - il secondo è opzionale e indica la base del sistema di rappresentazione numerica utilizzato nella stringa

```
parseInt("12")           // 12
```

```
parseInt("12abc")        // 12
```

```
parseInt("a12bc")        // NaN
```

```
parseInt("12.5")          // 12
```

```
parseInt("12", 8)         // 10,  $12_8 \Rightarrow 1 \cdot 8^1 + 2 \cdot 8^0$ 
```

Conversione di stringhe

- La funzione **parseFloat()** restituisce un valore numerico considerando l'eventuale virgola:

```
parseFloat("12")    // 12  
parseFloat("12.5")  // 12.5
```

- JSON.parse()** converte una stringa in un oggetto

```
JSON.parse('{ "nome": "Mario", "eta": 30 }')
```

- Date.parse()** converte una stringa in timestamp (millisecondi)

```
Date.parse("2024-01-01")
```

Array

- Gli **array** in JavaScript, o in altri linguaggi di programmazione, permettono di memorizzare molteplici valori in una singola variabile
- Fino ad ora abbiamo visto variabili contenitore di un singolo valore (numerico, booleano o stringa che sia), ma se dovessimo gestire tanti valori non sarebbe pratico creare tante variabili, una per ognuno di essi
- Gli array consentono di gestire una serie di valori riferendosi ad una sola variabile

Creare un array

- Il primo metodo è tramite il **costruttore Array**, dunque con l'uso di due keywords: “new” e “Array”

```
var frutti = new Array("mela", "pera", "arancia");
```

- I valori sono dichiarati all'interno delle parentesi tonde, e divisi l'uno dall'altro dalla virgola (,)
 - Il numero di valori che possono essere contenuti nell'array è illimitato
- Il secondo metodo, del tutto equivalente al primo e il più utilizzato, è tramite la **notazione letterale** e richiede l'uso delle parentesi quadre

```
var frutti = ["mela", "pera", "arancia"];
```

Accedere agli elementi di un array

- Per poter accedere a un valore salvato in un array si utilizza un **indice numerico** che rappresenta la **posizione** dell'elemento all'interno dell'array stesso
- Attenzione! La prima posizione è **0** e non 1!
- L'indice del valore interessato va indicato tra parentesi quadre

```
var frutti = ["mela", "pera", "arancia"];  
document.write(frutti[1]); // output => pera
```

- Se a un indice non dovesse corrispondere ad alcun valore verrà restituito “undefined”
- Un valore può essere sostituito in qualsiasi momento

```
frutti[1] = "fragola";
```

Array multidimensionali

- JavaScript consente di definire **array multidimensionali** o matrici, ovvero array che contengono array
- Ad esempio, quella che segue è la definizione di una matrice 3×3 di interi:

```
var matrice = [  
    [24, 13, 1],  
    [48, 92, 17],  
    [8, 56, 11]  
];
```

- Per accedere al numero 48 indicheremo le sue coordinate nel seguente modo:

```
var quarantotto = matrice[1][0];
```

Istruzioni condizionali

- Le istruzioni condizionali permettono di eseguire blocchi di codice in base alla verifica di una condizione
 - la verifica deve produrre come risultato vero o falso
- Si utilizzano le parole chiave **if**, **else if** e **else**

```
var voto = 75;
if (voto >= 80) {
    document.write("Ottimo");
} else if (voto >= 60) { // opzionale, ma ripetibile
    document.write("Discreto");
} else { // opzionale
    document.write("Insufficiente");
}
```

Istruzioni condizionali

- Quando si hanno delle istruzioni condizionali in cui abbiamo blocchi di codice differente in base al valore di una variabile è possibile utilizzare in alternativa il costrutto **switch/case**

```
var giorno = "martedì";
switch (giorno) {
  case "lunedì":
    console.log("Inizia la settimana");
    break; // da inserire alla fine di ogni case!
  case "venerdì":
    console.log("Quasi weekend!");
    break;
  default:
    console.log("Un giorno qualunque");
}
```

Istruzioni condizionali

- È possibile utilizzare una versione compatta del costrutto if/else per effettuare assegnazioni semplici
- Per questa versione di istruzione condizionale si utilizza l'operatore ternario ? :

```
var eta = 18;  
var messaggio = eta >=18 ? "Maggiorenne" : "Minorenne";  
document.write(messaggio);
```

Istruzioni iterative

- Servono per ripetere un blocco di istruzioni più volte
- È possibile effettuare una iterazione un numero definito o indefinito di volte stabilite a priori
- L'iterazione definita esegue un numero preciso di volte il blocco di istruzioni

```
var i;  
for (i = 0; i < 5; i++) {  
    document.write("Valore i: " + i);  
}
```

Istruzioni iterative

- L'iterazione indefinita può essere di due tipi:
 - il blocco di istruzioni può non essere mai eseguito

```
var i;  
for (i = 0; i < 5; i++) {  
    document.write("Valore i: " + i);  
}
```
 - il blocco di istruzioni viene eseguito almeno una volta

```
var i = 0;  
do {  
    document.write("Valore i: " + i);  
    i++;  
} while (i < 5);
```

Esecuzione

- JavaScript è un linguaggio imperativo le cui istruzioni vengono eseguite in maniera sequenziale e sincrona di default (anche se è possibile eseguirle asincrone)
- Quello che distingue JavaScript da molti altri linguaggi di programmazione (come C, Java o Python) è che **non ha una funzione main()** come punto di ingresso esplicito per l'esecuzione del programma
- Quando un file JavaScript viene eseguito, tutto il codice all'esterno di funzioni (**codice globale**) viene eseguito immediatamente (ossia quando lo script viene caricato) dalla prima all'ultima riga una volta sola

Le funzioni

- Le funzioni sono blocchi di codice riutilizzabile che effettuano una serie di operazioni e che eventualmente restituiscono un risultato
- Sono intercettate da un identificatore e possono accettare uno o più parametri di input
- È possibile usare una stessa funzione per ottenere risultati differenti in base ai valori presenti o scelti in fase di esecuzione
- Una funzione è eseguita nel momento in cui viene richiamata, a differenza dello script della pagina che viene eseguito al caricamento di quest'ultima
- La dichiarazione della funzione in sé non esegue il suo contenuto, ma semplicemente lo definisce

Sintassi delle funzioni

- Per definire una funzione si fa uso della keyword function
- Alla keyword function va seguito il nome personalizzato che si vuole assegnare alla funzione seguito dalle parentesi tonde, al cui interno si possono specificare i parametri, e le parentesi graffe {}, al cui interno va inserito il blocco di codice da eseguire
- Riguardo al nome della funzione, può essere di qualunque tipo, purché rispetti le regole JavaScript
 - Il nome personalizzato non può iniziare con un numero o un simbolo
 - I nomi personalizzati inoltre, non possono avere spazi tra le parole che lo compongono
 - Non si possono attribuire keywords ai nomi delle funzioni

Sintassi delle funzioni

```
function ciao() {  
    document.write("Ciao mondo!");  
}
```

```
function somma(x, y) { // attenzione alla virgola  
    document.write(x+y);  
}  
ciao();  
somma(5, 3);
```

Alert, prompt e confirm

- **Alert**, **prompt** e **confirm** servono per programmare la comparsa di avvisi o messaggi popup
- La differenza tra le tre tipologie sta nel tipo di interazione che i visitatori dovranno compiere alla loro comparsa
- I popup a comparsa sono generati dal browser in uso e non realizzati dal programmatore
- Ognuno di questi metodi è accompagnato da un messaggio
 - alert è lo strumento per la comparsa di avvisi/messaggi specifici all'utente
 - confirm è simile ad alert, ma sottopone l'utente a una scelta
 - prompt richiede non solo l'interazione dell'utente, ma anche di immettere un valore testuale nel campo della finestra popup

La manipolazione del DOM

- DOM (Document Object Model) è la rappresentazione ad albero sintattico di una pagina HTML
- Ogni elemento HTML è dunque un nodo
- JavaScript può effettuare le seguenti operazioni sugli elementi del DOM: leggere, modificare, creare, eliminare

```
<html>  
  <body>  
    <h1>Titolo</h1>  
    <p>Paragrafo</p>  
  </body>  
</html>
```



```
document  
└── html  
    └── body  
        ├── h1  
        └── p
```

La manipolazione del DOM

- **Selezionare elementi del DOM**

```
document.getElementById("titolo")
```

```
document.getElementsByClassName("box")
```

```
document.getElementsByTagName("p")
```

```
document.querySelector(".box")
```

```
document.querySelectorAll("p")
```

- **Esempio:**

```
<h1 id="titolo">Ciao</h1>
```



```
var elemento = document.getElementById("titolo");
```

`document` rappresenta la pagina, è l'elemento radice del DOM

La manipolazione del DOM

- Modificare il contenuto del DOM

`innerText`

`innerHTML`

`textContent`

- Esempio:

```
var titolo = document.getElementById("titolo");
```

```
titolo.innerText = "Nuovo Titolo";
```

```
titolo.innerHTML = "<em>Nuovo Titolo</em>";
```

La manipolazione del DOM

- Modificare attributi di elementi HTML del DOM

`setAttribute`

`getAttribute`

- Esempio:

```

```



```
var img = document.getElementById("foto");  
img.setAttribute("src", "immagine.jpg");
```

La manipolazione del DOM

- Creare nuovi elementi nel DOM

- creare l'elemento
- aggiungere il contenuto all'elemento creato
- inserirlo nel DOM

- Esempio:

```
var nuovoParagrafo = document.createElement("p");  
nuovoParagrafo.innerText = "Paragrafo creato con JS";  
document.body.appendChild(nuovoParagrafo);
```

- Rimuovere elementi nel DOM

```
var titolo = document.getElementById("titolo");  
titolo.remove();
```

```
titolo.parentNode.removeChild(titolo);
```

La manipolazione del DOM

- Modificare lo stile di elementi HTML del DOM accedendo alla proprietà style dell'elemento

- Esempio:

```
var titolo = document.getElementById("titolo");  
titolo.style.color = "blue";
```

- Attenzione! Le proprietà CSS con trattino in JavaScript diventano camelCase:

```
background-color ==> backgroundColor
```

La manipolazione del DOM

- Effettuare operazioni al verificarsi di eventi scaturiti dall'interazione dell'utente con elementi del DOM
- Alcuni eventi:
 - click
 - mouseover
 - keydown
- Esempio:

```
<button id="btn">Cliccami</button>
```

```
var bottone = document.getElementById("btn");  
bottone.addEventListener("click", function() {  
    alert("Hai cliccato!");  
});
```